

FocusLab never sees a Spotify password. It asks Spotify for permission to act on the user's behalf, and is handed a short-lived token instead. There is no API key anywhere in the system — every credential here is an OAuth artifact.

THE FLOW, END TO END

- 1 User clicks Connect → GET /spotify/login**
Backend mints `secrets.token_urlsafe(32)` as the CSRF state.
- 2 State is stored server-side, 10-minute TTL**
Not a cookie — so the flow survives a `localhost → 127.0.0.1` hop and needs no browser at all.
- 3 Redirect to `accounts.spotify.com/authorize`**
Carrying `client_id`, `redirect_uri`, the 8 scopes, and `state`.
- 4 User approves on Spotify's own page**
Credentials are typed into Spotify, never into FocusLab.
- 5 Spotify redirects to GET /spotify/callback?code=...&state=...**
`claim_login_state()` matches in constant time and consumes it — one state, one callback. Mismatch → `/?spotify=error`.
- 6 Backend exchanges the code, server-to-server**
POST `/api/token` authenticated with HTTP Basic (`client_id` + `client_secret`). This is why it cannot happen in the browser: the secret would be exposed.
- 7 Tokens saved to SQLite, row id=1**
`access_token`, `refresh_token`, `expires_at`. The code is single-use and dies here.

TWO CREDENTIALS, TWO DESTINATIONS

CREDENTIAL	GOES TO	HOW	WHEN
Client ID + Secret	<code>accounts.spotify.com</code>	HTTP Basic	Exchange & refresh only
Access token	<code>api.spotify.com</code>	Bearer	Every API call

The secret never reaches `api.spotify.com`; the token never reaches `accounts.spotify.com`. Neither is an API key — a key is static and unscoped, these expire, carry scopes, and the user can revoke them.

TOKEN LIFECYCLE

Access token	~1 hour
Refreshed	60 s early, on demand
Refresh token	Reused if Spotify sends none
Refresh rejected	Row deleted → reconnect

Browser player. The Web Playback SDK runs in the page, so `/spotify/token` hands it the raw access token — the only token that reaches the browser. It re-fetches on expiry rather than caching.

WHY STATE, AND WHY PLAIN HTTP

```
browser → accounts.spotify.com/authorize HTTPS
Spotify → 127.0.0.1:8000/spotify/callback HTTP ← the leg state protects
backend → accounts.spotify.com/api/token HTTPS
backend → api.spotify.com HTTPS
```

State stops authorization-code injection: an attacker feeding you *their* code so your app silently links to *their* account. OAuth has no built-in defence on the redirect — RFC 6749 §10.12 requires the client to add one. HTTPS would not help; the forged request is perfectly valid TLS. **Plain HTTP** is correct here because the callback is loopback: no CA issues certificates for `127.0.0.1`, and the traffic never touches a network. RFC 8252 prescribes exactly this for native apps, and it is the only HTTP Spotify still accepts.

SCOPES REQUESTED

playlist-read-private playlist-read-collaborative user-read-private user-read-email user-read-playback-state
user-modify-playback-state user-read-recently-played streaming

SPOTIFY — OAUTH · `apis/spotify/OAuth_Logic.py`

GET	<code>/spotify/login</code>	Starts the OAuth flow by redirecting the user to Spotify
GET	<code>/spotify/callback</code>	Where Spotify sends the user back, and where the code is exchanged
GET	<code>/spotify/status</code>	Reports whether Spotify has ever been connected
GET	<code>/spotify/token</code>	Hands the access token to the browser's Web Playback SDK

SPOTIFY — READS · `apis/spotify/routes/get.py`

GET	<code>/spotify/player</code>	What is playing right now, trimmed to what the UI needs
GET	<code>/spotify/devices</code>	Every Spotify app this account is signed in to
GET	<code>/spotify/search</code>	Track search, used when adding songs to a queue
GET	<code>/spotify/recently-played</code>	The last songs played on this account

SPOTIFY — PLAYBACK · `routes/put.py` & `post.py`

PUT	<code>/spotify/pause</code>	Stops playback, keeping the place in the queue
PUT	<code>/spotify/resume</code>	Carries on from wherever playback was paused
POST	<code>/spotify/next</code>	Jumps forward to the next song
POST	<code>/spotify/previous</code>	Jumps back to the previous song

QUEUES · `apis/queues/routes/`

GET	<code>/queues</code>	Lists every saved queue, newest first
GET	<code>/queues/{queue_id}</code>	Returns one queue together with all of its songs
POST	<code>/queues</code>	Creates a new empty queue
PATCH	<code>/queues/{queue_id}</code>	Renames a queue
DELETE	<code>/queues/{queue_id}</code>	Deletes a queue along with its songs
POST	<code>/queues/{queue_id}/play</code>	Plays the whole queue on Spotify from a chosen song
POST	<code>/queues/{queue_id}/tracks</code>	Appends a song to the end of the queue
DELETE	<code>/queues/{id}/tracks/{track_id}</code>	Removes one song and closes the gap it leaves

TO-DO NOTES · `apis/todo/routes/`

GET	<code>/notes</code>	Lists every saved note, most recently edited first
GET	<code>/notes/{note_id}</code>	Returns one note
POST	<code>/notes</code>	Creates a new note, blank unless a title/content is sent
PATCH	<code>/notes/{note_id}</code>	Edits a note's title and/or content
DELETE	<code>/notes/{note_id}</code>	Deletes a note

API KEYS · `apis/Retrieving_Keys/routes/`

GET	<code>/keys/status</code>	Reports which keys are saved, without ever revealing the secret
POST	<code>/keys</code>	Saves the Spotify client id and secret the user pasted in

Backend layout. One package per service under `apis/`, one file per HTTP method under each `routes/`. All Spotify OAuth — the flow *and* the token lifecycle — lives in `OAuth_Logic.py`; `core.py` holds Web API plumbing and depends on it for a valid token, never the reverse.

No auth on FocusLab's own API. The 27 routes above are unauthenticated — CORS is the only gate, and CORS is a browser rule. That is deliberate: FocusLab is built as a **desktop app**, one install per user, backend on localhost. Single-row tables and user-supplied keys follow from the same decision.